

Strings

Every Java interview asks about `String`, and most of the questions come from one fact: a `String` cannot be changed after it is made. This part is those questions, with the answer first and the proof after it.

1. What is String in Java?

`String` is a class in `java.lang` that holds a sequence of characters. It is not a primitive type.

```
public class Main {  
    public static void main(String[] args) {  
        String name = "Rohit";  
  
        System.out.println(name.length());  
        System.out.println(name.getClass().getName());  
        System.out.println(name instanceof Object);  
    }  
}
```

Output:

```
5  
java.lang.String  
true
```

Three properties matter, and every later question depends on one of them:

Property	What it means
<code>final</code> class	Nothing can extend <code>String</code>
Immutable	The characters never change after construction
Pooled	Identical literals share one object

It implements `CharSequence`, `Comparable<String>` and `Serializable`, which is why it works in a `TreeMap`, in a `switch`, and as a sorted key.

It looks like a primitive and is not one. The `+` operator is the only place Java overloads an operator for a class, and the assignment syntax `String s = "hello"` hides an object creation. Both are conveniences, not evidence.

2. What is String immutability, and why is String immutable?

Immutable means the value cannot change after the object is created. Every method that appears to modify a String returns a new one instead.

```
public class Main {  
    public static void main(String[] args) {  
        String s = "hello";  
  
        s.concat(" world");  
        s.toUpperCase();  
        System.out.println(s);  
  
        String t = s.concat(" world");  
        System.out.println(t);  
    }  
}
```

Output:

```
hello  
hello world
```

The first two calls do work and throw the result away. This is the most common beginner bug in the language, and it is the fastest way to show that a String never changes.

Why it was designed that way, in the order an interviewer expects:

Reason	Detail
The pool needs it	Many variables share one object. If one could edit it, all would change
Security	A file path or URL checked by one method cannot be altered before another uses it
Thread safety	Nothing can change, so no synchronization is ever needed
Cached hash	The hash can never go stale, so it is computed once

The cached hash is the reason `String` is the default choice for a `HashMap` key. A mutable key whose hash changed after insertion would be lost in the map.

The cost is garbage. Building a string in a loop creates a new object each time, which is why `StringBuilder` exists.

3. What are String literals?

A literal is a value written directly in double quotes. Literals are placed in the String Pool by the compiler and the JVM, so identical ones share one object.

```
public class Main {  
    public static void main(String[] args) {  
        String a = "hello";  
        String b = "hello";  
        String c = "hel" + "lo";  
        String d = new String("hello");  
  
        System.out.println(a == b);  
        System.out.println(a == c);  
        System.out.println(a == d);  
    }  
}
```

Output:

```
true  
true  
false
```

c is **true** because of **constant folding**. Both halves are known at compile time, so the compiler joins them and the result is the same literal. Nothing is joined at run time.

Change one half to a variable and the answer changes:

```
String part = "hel";  
String e = part + "lo";    // built at run time, a new object  
System.out.println(a == e); // false
```

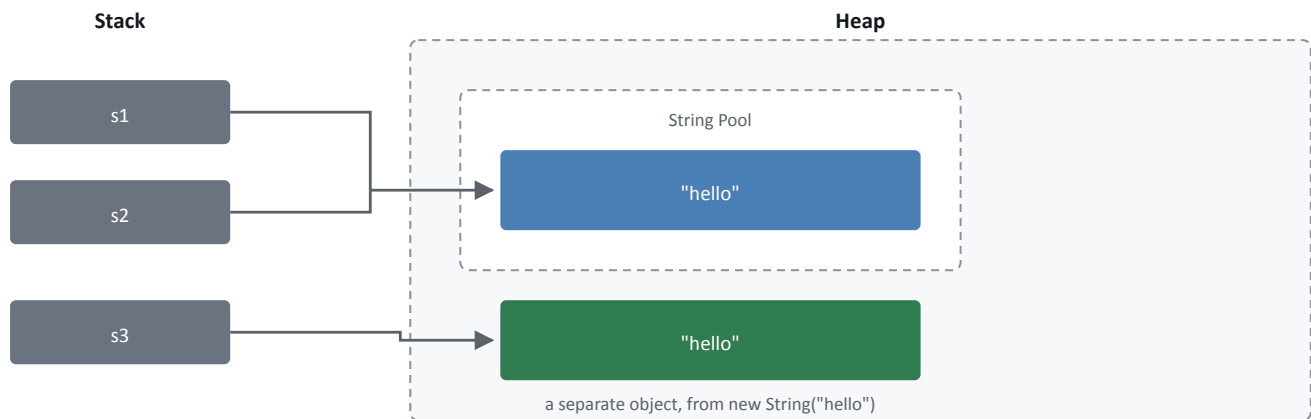
Unless that variable is **final**, in which case it is a compile-time constant again and folding returns:

```
final String part = "hel";  
String f = part + "lo";  
System.out.println(a == f); // true
```

This is why `==` is unreliable for Strings. The answer depends on where the value came from, which is not something the reading code can see.

4. What is the String Pool?

The String Pool is a cache the JVM keeps of String objects, so that identical literals are stored once and shared.



Two literals with the same characters are one object. The same characters passed to `new` are a second object with equal content.

The pool exists because strings are the most common object in most programs, and most of them repeat. Sharing them saves a large amount of memory for nothing.

```
String a = "hello";
String b = "hello";    // no new object created
String c = new String("hello"); // a new object, always
```

`new String(...)` always creates an object and always skips the pool. That is its only real use in modern code: proving a point in an interview.

5. Where is the String Pool located?

In the heap, since Java 7. Before that it was in PermGen.

Java version	Pool location
6 and earlier	PermGen, a fixed-size area outside the heap
7	Moved into the heap
8 and later	Still the heap. PermGen itself was replaced by Metaspace

It moved because PermGen was a fixed size set at startup, and an application that interned many strings filled it and died with `OutOfMemoryError: PermGen space`. Tuning it meant guessing a number before running the program.

Two consequences follow from the move, and both are worth saying:

Pooled strings can now be garbage collected. In the heap they are ordinary objects, so a string that nothing references any more is collected like anything else. In PermGen it stayed forever.

Running out now looks like a normal heap problem, with the usual `OutOfMemoryError: Java heap space` and the usual tools, rather than a separate area with its own flag.

The pool is a fixed-size hash table, and its bucket count can still be raised when an application interns very many distinct strings:

```
-XX:StringTableSize=60013
```

6. What is the difference between `==` and `equals()` ?

`==` compares references and asks whether two variables point at the same object. `equals()` compares content and asks whether two strings hold the same characters.

```
public class Main {
    public static void main(String[] args) {
        String a = "hello";
        String b = "hello";
        String c = new String("hello");

        System.out.println(a == b);
        System.out.println(a == c);
        System.out.println(a.equals(c));
    }
}
```

Output:

```
true
false
true
```

	<code>==</code>	<code>equals()</code>
Compares	The reference	The characters
Defined by	The language	<code>String</code> , overriding <code>Object</code>
<code>"hello"</code> and <code>"hello"</code>	<code>true</code> , the pool shares them	<code>true</code>
<code>"hello"</code> and <code>new String("hello")</code>	<code>false</code>	<code>true</code>

Always use `equals()` for Strings. `==` sometimes gives the right answer, which makes it worse than always being wrong; the bug appears the day a value arrives from a file or a network call instead of a literal.

Put the literal on the left to avoid a null check.

```
if ("admin".equals(role)) { ... }    // safe when role is null
if (role.equals("admin")) { ... }    // NullPointerException when role is null
```

7. What is the difference between `trim()` and `strip()` ?

Both remove whitespace from both ends. `trim()` removes characters at or below `U+0020`; `strip()` removes anything Unicode calls whitespace.

```
public class Main {
    public static void main(String[] args) {
        String ascii  = " hello ";
        String unicode = "\u2000hello\u2000";

        System.out.println "[" + ascii.trim() + "];
        System.out.println "[" + ascii.strip() + "];
        System.out.println(unicode.trim().length());
        System.out.println(unicode.strip().length());
    }
}
```

Output:

```
[hello]
[hello]
7
5
```

The last two lines are the difference. `trim` left seven characters, because `U+2000` is above `U+0020` and it does not count as whitespace. `strip` removed both and left five.

	<code>trim()</code>	<code>strip()</code>
Added in	Java 1.0	Java 11
Removes	Characters <code><= U+0020</code>	Anything <code>Character.isWhitespace</code> accepts
Unicode aware	No	Yes
One-sided version	None	<code>stripLeading()</code> , <code>stripTrailing()</code>

Use `strip()` in new code. `trim()` is kept for compatibility and predates Unicode. On input that is only ASCII the two behave identically, which is why the difference stays hidden until real user data arrives.

`isBlank()` arrived with `strip()` and answers the related question:

```
" ".isEmpty() // false, it has three characters
" ".isBlank() // true, it is only whitespace
```

8. What does `equals()` do?

`equals()` returns true when the other object is a String of the same length holding the same characters, in the same case.

```
public class Main {
    public static void main(String[] args) {
        String a = "Hello";

        System.out.println(a.equals("Hello"));
        System.out.println(a.equals("hello"));
        System.out.println(a.equals("Hello "));
        System.out.println(a.equals(null));
        System.out.println(a.equals(42));
    }
}
```

Output:

```
true
false
false
false
false
```

Inside, it checks the cheap things first: the same reference is true at once, a non-String argument is false at once, a different length is false at once, and only then are the characters compared.

It never throws. A `null` argument returns false rather than failing, so the null problem is only ever with the object the method is called on.

Overriding `equals` means overriding `hashCode`. `String` does both, which is what makes it a safe `HashMap` key. Any class that overrides one and not the other breaks in a hash-based collection.

9. What does `equalsIgnoreCase()` do?

The same comparison as `equals()`, ignoring case.

```
public class Main {
    public static void main(String[] args) {
        System.out.println("ADMIN".equals("admin"));
        System.out.println("ADMIN".equalsIgnoreCase("admin"));
        System.out.println("ADMIN".equalsIgnoreCase("Admin"));
        System.out.println(" admin".equalsIgnoreCase("admin"));
    }
}
```

Output:

```
false
true
true
false
```

It ignores case and nothing else. Whitespace, accents and length all still count, which the fourth line shows.

Prefer it to lower-casing both sides. `a.toLowerCase().equals(b.toLowerCase())` allocates two strings and depends on the default locale, where a Turkish locale turns `I` into a dotless `ı` and the comparison fails. `equalsIgnoreCase` does neither.

10. What does `compareTo()` return?

An `int`: negative when this string sorts first, zero when they are equal, and positive when it sorts later.

```
public class Main {
    public static void main(String[] args) {
        System.out.println("apple".compareTo("banana"));
        System.out.println("apple".compareTo("apple"));
        System.out.println("banana".compareTo("apple"));
        System.out.println("apple".compareTo("app"));
        System.out.println("Apple".compareTo("apple"));
    }
}
```

Output:

```
-1
0
1
2
-32
```

The number is not arbitrary. At the first position where the strings differ it is the difference between the two characters; when one is a prefix of the other it is the difference in length.

Comparison	Result	Why
"apple" to "banana"	-1	'a' is 97, 'b' is 98
"apple" to "app"	2	No difference found, so 5 minus 3
"Apple" to "apple"	-32	'A' is 65, 'a' is 97

Only the sign is contractual. Code should test `< 0`, `== 0` or `> 0` and never the exact number, which other `Comparable` types do not promise.

Capitals sort before lower case, because that is their order in the character set. This is why an unsorted-looking list of names is usually a list sorted exactly as asked.

`compareTo` is what `TreeSet`, `TreeMap` and `Collections.sort` use, and for `String` it is consistent with `equals`: it returns zero exactly when `equals` returns true.

11. What does `compareToIgnoreCase()` do?

The same ordering, with case ignored.

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Apple".compareTo("apple"));
        System.out.println("Apple".compareToIgnoreCase("apple"));
        System.out.println("apple".compareToIgnoreCase("Banana"));
    }
}
```

Output:

```
-32
0
-1
```

The third line is the point. With `compareTo`, `"Banana"` sorts before `"apple"` because `'B'` is 66 and `'a'` is 97. Ignoring case gives the order a person expects.

```
List<String> names = new ArrayList<>(List.of("banana", "Cherry", "apple"));

names.sort(String::compareTo);
System.out.println(names);

names.sort(String::compareToIgnoreCase);
System.out.println(names);
```

Output:

```
[Cherry, apple, banana]
[apple, banana, Cherry]
```

The first list looks wrong to a reader and is exactly what was asked for: every capital letter sorts before every lower-case one. `String.CASE_INSENSITIVE_ORDER` is the second comparator with a name.

It is not consistent with `equals`. `"Apple".compareToIgnoreCase("apple")` is zero while `equals` is false, and a `TreeSet` built on that comparator therefore treats the two as one entry and keeps only the first.

12. `replace()` against `replaceAll()` and `replaceFirst()`

`replace()` works on plain text and replaces every match. `replaceAll()` and `replaceFirst()` take a regular expression.

```
public class Main {
    public static void main(String[] args) {
        String version = "1.4.2";

        System.out.println(version.replace(".", "-"));
        System.out.println(version.replaceAll(".", "-"));
        System.out.println(version.replaceFirst("\\.", "-"));
    }
}
```

Output:

```
1-4-2
-----
1-4.2
```

The second line is the trap. In a regular expression `.` means any character, so every one of the five characters is replaced.

Method	Pattern is	Replaces
<code>replace()</code>	Literal text	Every occurrence
<code>replaceAll()</code>	A regular expression	Every match
<code>replaceFirst()</code>	A regular expression	The first match only

The names are misleading. `replace` sounds like one and does all; `replaceAll` sounds like the same thing and interprets its argument completely differently.

Use `replace()` unless a pattern is actually needed. It is faster, because nothing is compiled, and it cannot surprise anyone with a metacharacter.

The replacement text has metacharacters too. In the regex versions `$1` is a captured group and `\` escapes, so text from outside the program needs `Matcher.quoteReplacement`.

```
"cost: 100".replace("100", "$5");           // cost: $5
"cost: 100".replaceAll("100", "$5");        // IndexOutOfBoundsException: No group 5
```

13. What is String interning?

Interning puts a string into the pool, or returns the pooled object if one with the same characters is already there.

```
public class Main {
    public static void main(String[] args) {
        String a = "hello";
        String b = new String("hello");
        String c = b.intern();

        System.out.println(a == b);
        System.out.println(a == c);
        System.out.println(a.equals(b));
    }
}
```

Output:

```
false
true
true
```

`b` is its own object. `intern()` does not change it; it returns the pooled instance, which is the same object `a` points at.

Literals are interned automatically. `intern()` is only for strings built at run time, from a file, a request or a database column.

It is rarely worth calling. The case that justifies it is reading many rows where one column repeats heavily, such as a country code on a million records: one object instead of a million.

```
String country = row.getString("country").intern();
```

A million rows carrying thirty distinct codes then hold a million references to thirty objects, rather than a million objects.

The pool is shared by the whole JVM, and it is a fixed-size hash table. That makes interning a decision about global state rather than about one piece of code, which is the real reason to be careful with it.

Against that, three costs:

Cost	Detail
Time	Every call hashes the string and searches the pool
A shared table	The pool is fixed-size, and heavy use means long buckets
Usually pointless	On a value that rarely repeats it only adds work

Measure before using it. Interning is a memory optimization with a CPU price, and applying it without evidence usually makes a program slower rather than smaller.

The pieces together

Question	The short answer
What is it	A final, immutable class in <code>java.lang</code>
Why immutable	Pool sharing, security, thread safety, cached hash
What is the pool	A cache so identical literals are one object
Where is the pool	The heap, since Java 7
<code>==</code> or <code>equals</code>	<code>equals</code> for content, always
<code>trim</code> or <code>strip</code>	<code>strip</code> , unless the code must run before Java 11
What <code>compareTo</code> returns	A sign, not a number worth reading
<code>replace</code> or <code>replaceAll</code>	<code>replace</code> for text, <code>replaceAll</code> for a pattern
What interning is	Putting a run-time string into the pool, on purpose

For example, imagine reading a million-row export. Each row has a country code, and building them with `new String(...)` or by concatenation produces a million objects holding perhaps thirty distinct values. Calling `intern()` on that one column collapses them onto the pooled instances, which live in the heap and are collected when the import finishes. Comparing them with `==` would still be wrong, because the next release might read the same column from a different source.

The habit worth building is to reach for `equals` without thinking about it, and to treat every question about `==` as a question about where the string came from. Almost every String puzzle in an interview is that question in disguise.